Linux Practical with Solution

Practical 1: Installation and Basic Graphical Environment of Red Hat Enterprise Linux

Aim

To install **Red Hat Enterprise Linux (RHEL)** using **VirtualBox/VMware** and study the Linux graphical environment, display/login manager, desktop environment, directory structure, and file systems.

Software/Hardware Requirements

- Computer/Laptop with at least 8 GB RAM
- Oracle VirtualBox or VMware Workstation
- RHEL ISO image
- Minimum 40 GB free disk space
- Internet connection
- Keyboard and mouse

Part A: Installing Red Hat Enterprise Linux Using VirtualBox

Objective

To create a virtual machine and install RHEL on it.

Procedure

Step 1: Install VirtualBox

1. Download and install **Oracle VirtualBox**.
2. Open VirtualBox.
3. Click **New** to create a virtual machine.

Step 2: Create Virtual Machine

Enter the following details:

Setting	Recommended Value
Name	RHEL-Linux

Type	Linux
Version	Red Hat (64-bit)
RAM	4 GB or more
CPU	2 processors or more
Hard Disk	40 GB
Disk Type	VDI
Storage	Dynamically allocated

Click **Finish**.

Step 3: Attach RHEL ISO

1. Select the newly created VM.
2. Go to **Settings** → **Storage**.
3. Select the virtual optical drive.
4. Choose the downloaded **RHEL ISO** file.
5. Click **OK**.

Step 4: Start Installation

1. Click **Start**.
2. Select **Install Red Hat Enterprise Linux**.
3. Select the required language and keyboard layout.
4. Configure:
    - Installation destination
    - Time and date
    - Network
    - Software selection
    - Root password
    - User account

Step 5: Software Selection

Select a graphical environment such as:

Server with GUI / Workstation, depending on the RHEL version and available subscription/software repositories.

Step 6: Disk Configuration

Select the virtual hard disk.

For a beginner installation, choose:

Automatic partitioning

The installer will create the required partitions/logical volumes.

Step 7: Create User

Create a normal user, for example:

Username: student
Password: ********

Set a strong password.

Step 8: Begin Installation

Click **Begin Installation**.

Wait until the installation is completed.

Step 9: Reboot

After installation:

1. Click **Reboot System**.
2. Remove/eject the RHEL ISO if necessary.
3. Allow the virtual machine to boot from its virtual disk.

Result

RHEL is successfully installed and the system boots into the Linux environment.

Part B: Linux Graphical Environment

1. Display/Login Manager

Definition

A **Display Manager** provides the graphical login screen through which a user selects or enters their account and starts a graphical session.

In RHEL, **GDM (GNOME Display Manager)** is commonly used with the GNOME desktop.

Practical: Identify the Display Manager

Open **Terminal** and execute:

systemctl status gdm

Expected Output

You may see information similar to:

● gdm.service - GNOME Display Manager

Loaded: loaded
Active: active (running)

This indicates that GDM is running.

Check the Current Display Manager

Execute:

systemctl status display-manager

Another useful command is:

ls -l /etc/systemd/system/display-manager.service

Explanation

The display manager:

1. Displays the graphical login screen.
2. Authenticates the user.
3. Starts the user's graphical session.
4. Provides session selection when multiple desktop environments are installed.
5. Handles graphical login/logout operations.

Result

The display/login manager is identified successfully.

Part C: Desktop Environment

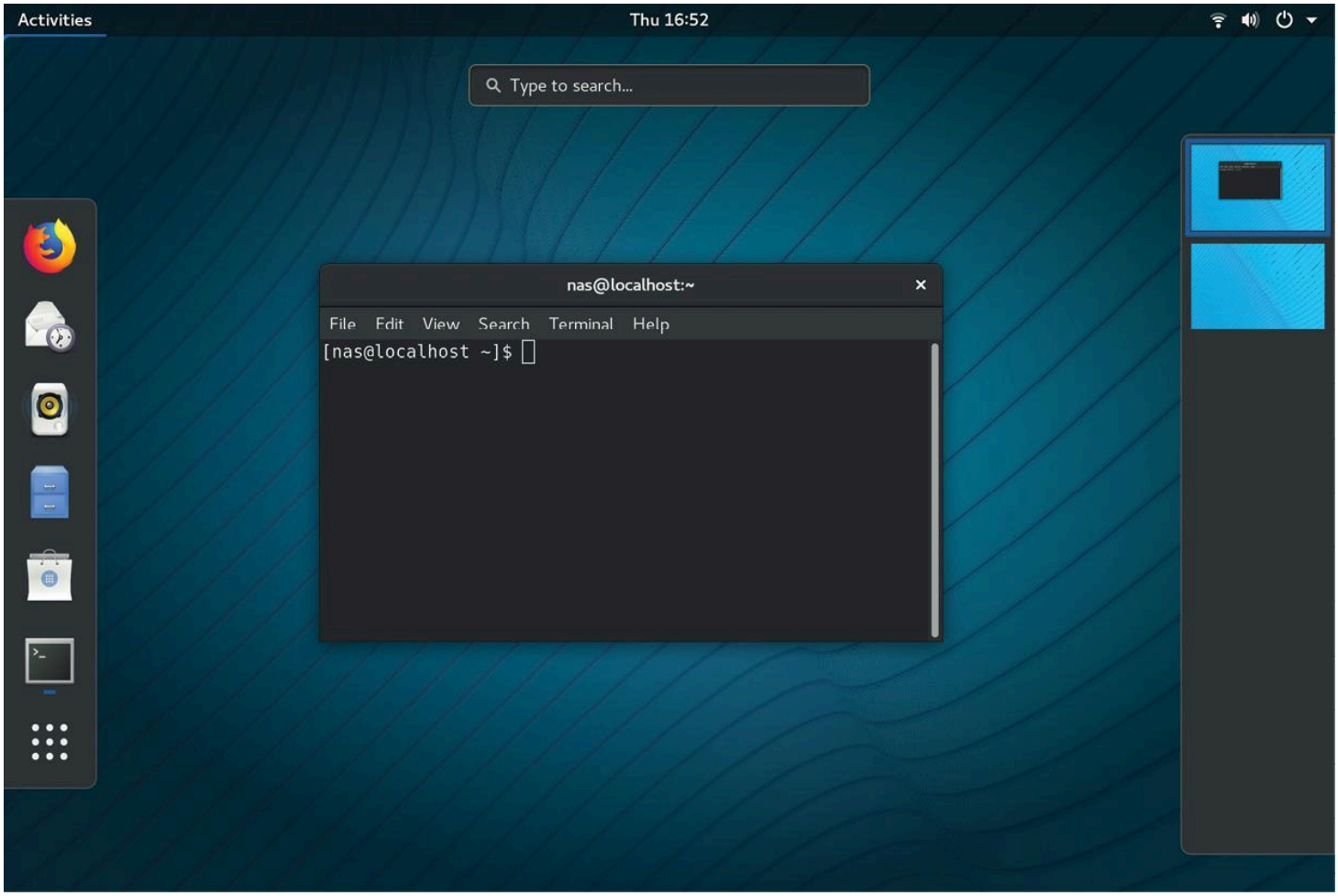
2. Study of GNOME Desktop Environment

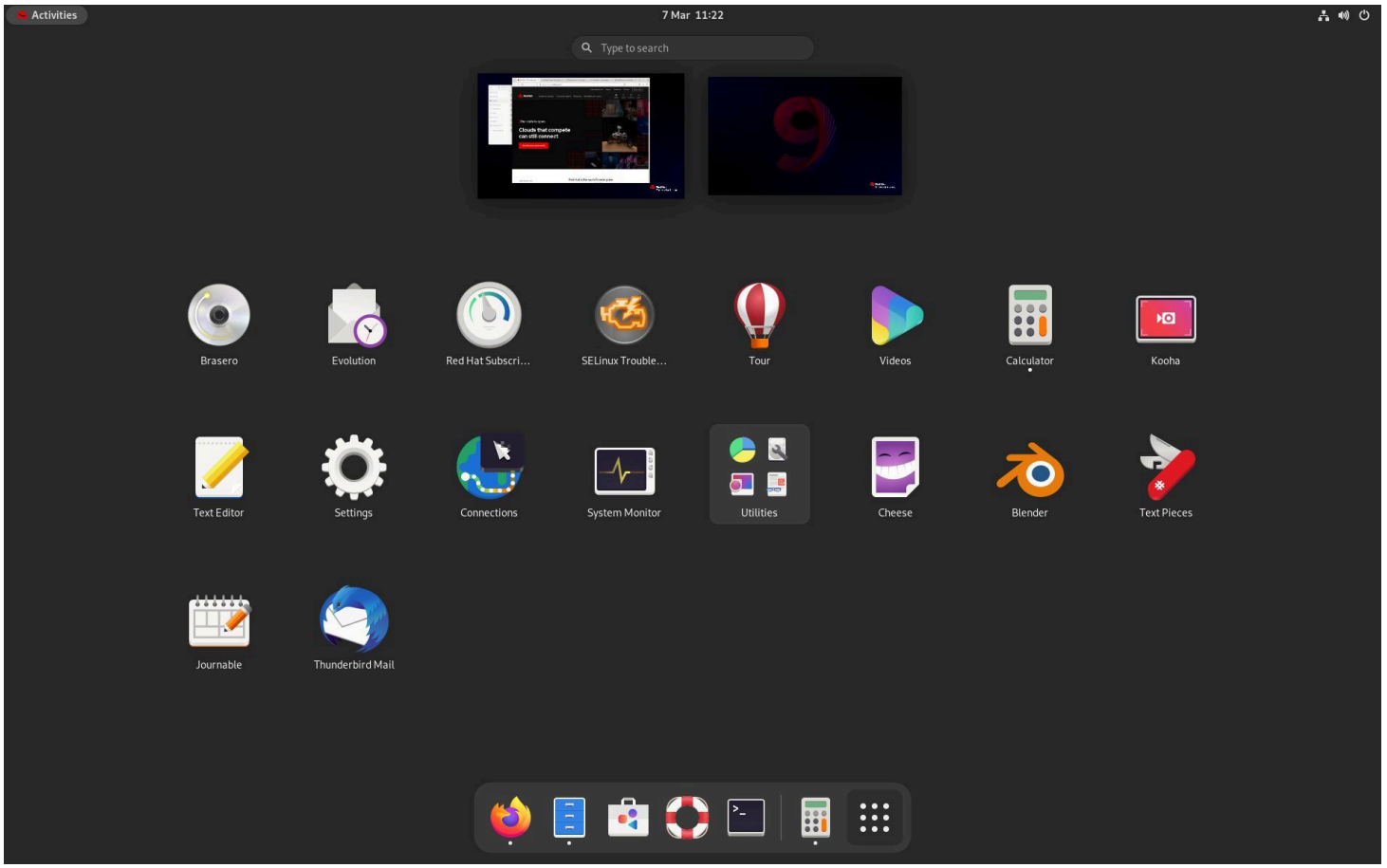
Definition

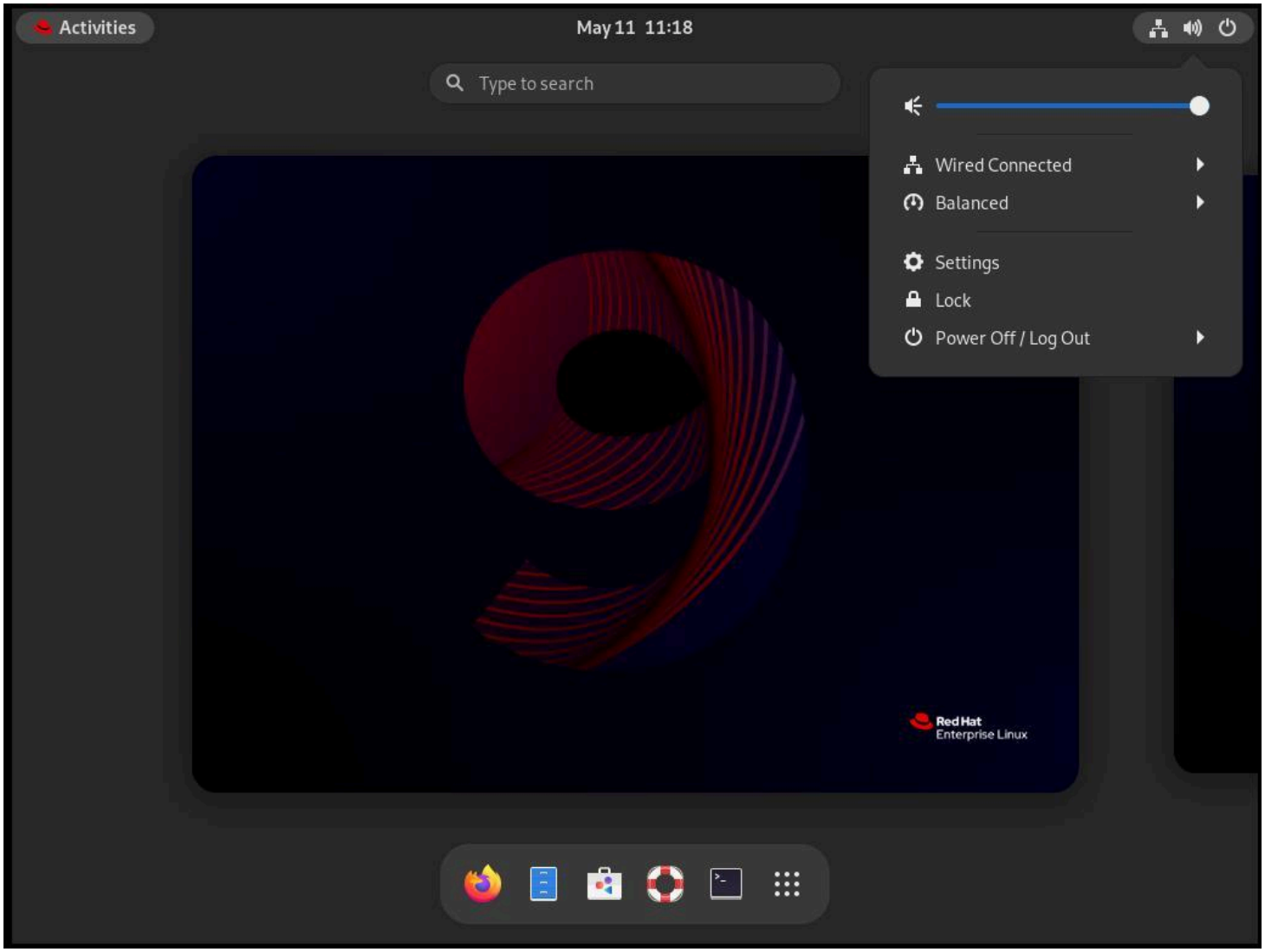
A **Desktop Environment (DE)** provides the graphical interface through which users interact with Linux.

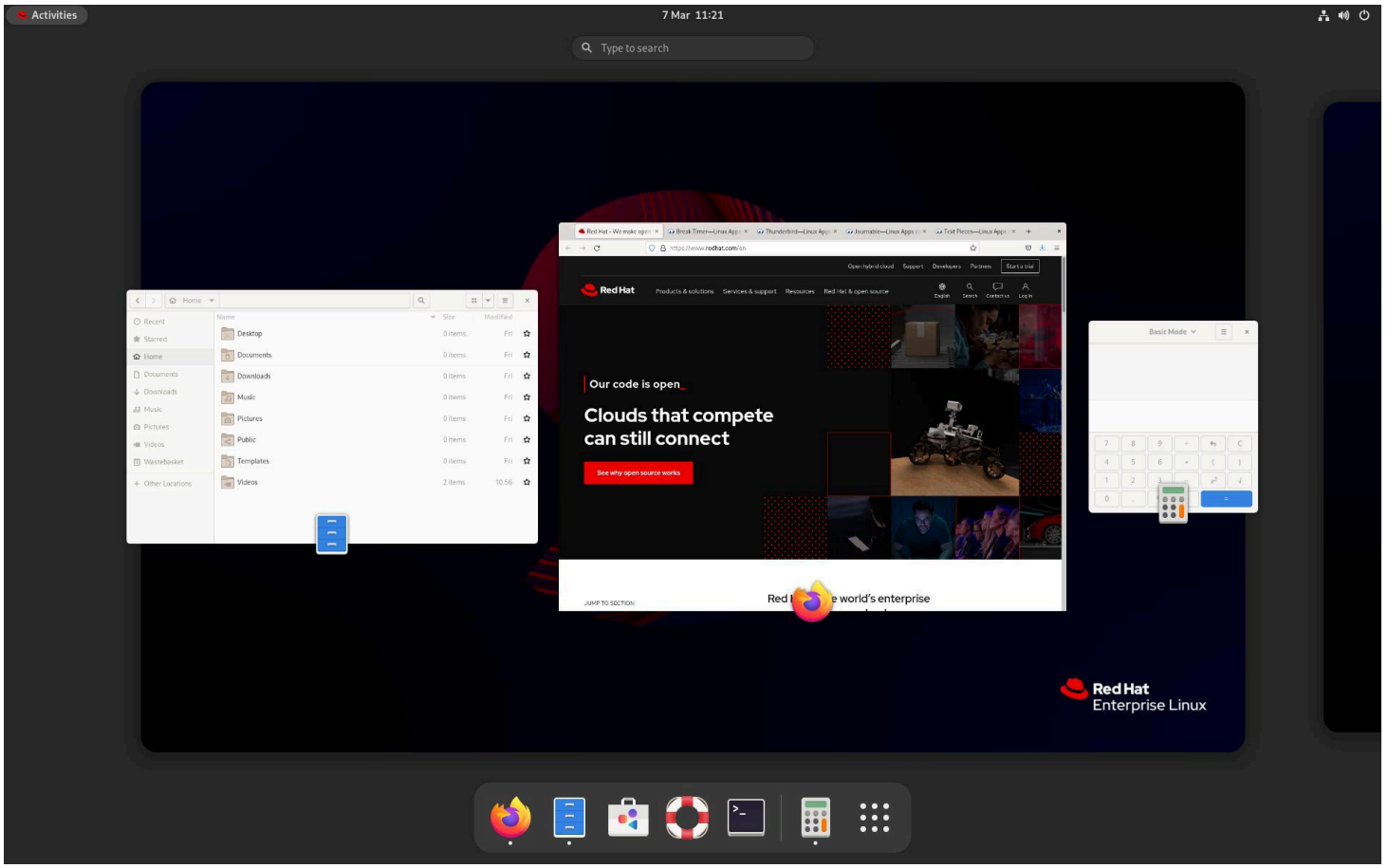
RHEL commonly uses the **GNOME Desktop Environment**.

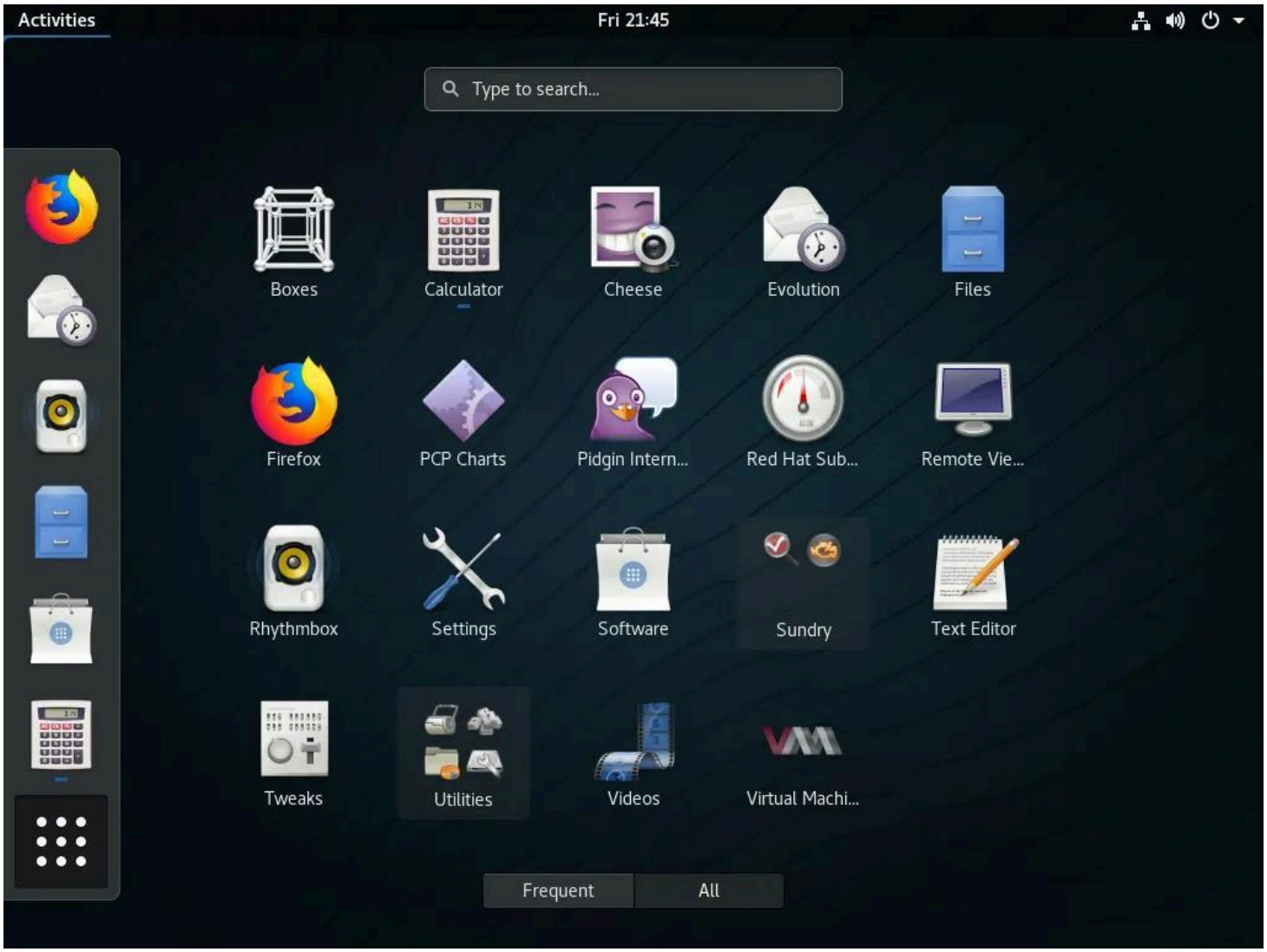

Activities
Type to search...
Red Hat
Enterprise Linux


Activities
Thu 16:52
Type to search...
nas@localhost:~
File Edit View Search Terminal Help
[nas@localhost ~]$


Activities
7 Mar 11:22
Type to search
Brasero
Evolution
Red Hat Subscri...
SELinux Trouble...
Tour
Videos
Calculator
Kooha
Text Editor
Settings
Connections
System Monitor
Utilities
Cheese
Blender
Text Pieces
Journable
Thunderbird Mail


Activities
May 11 11:18
Type to search
Wired Connected
Balanced
Settings
Lock
Power Off / Log Out
Red Hat
Enterprise Linux


Activities
7 Mar 11:21
Type to search
Our code is open_
Clouds that compete
can still connect
See why open source works
Red Hat
Enterprise Linux



Main Components

The GNOME environment generally includes:

- Applications menu
- Activities Overview
- Desktop
- Application launcher
- File Manager
- Terminal
- Settings
- System status area
- Notification area

Practical: Identify Desktop Environment

Open Terminal and execute:

echo $XDG_CURRENT_DESKTOP

Possible output:

GNOME

You can also execute:

echo $DESKTOP_SESSION

Check Linux Version

cat /etc/redhat-release

or:

cat /etc/os-release

Check Kernel Version

uname -r

Result

The graphical desktop environment and Linux system information are successfully identified.

Part D: Linux Directory Structure

3. Study of Linux Directory Structure

Linux follows a hierarchical directory structure. The top-level directory is represented by:

/

This is called the **root directory**.

Important Linux Directories

Directory	Purpose
`/`	Root of the entire file system
`/bin`	Essential user commands; on modern RHEL systems this may be linked to `/usr/bin`
`/boot`	Bootloader files and Linux kernel
`/dev`	Device files
`/etc`	System configuration files
`/home`	Users' home directories
`/lib`	Essential libraries; commonly linked into `/usr/lib` on modern systems

`/media`	Mount points for removable media
`/mnt`	Temporary mount point
`/opt`	Optional/add-on software
`/proc`	Virtual file system containing process/kernel information
`/root`	Home directory of the root user
`/run`	Runtime system data
`/sbin`	Essential system administration commands; commonly linked into `/usr/sbin`
`/tmp`	Temporary files
`/usr`	User applications, libraries, and shared resources
`/var`	Variable data such as logs and caches

Practical: Display Directory Structure

Open Terminal:

cd /

Display the contents:

ls

For detailed information:

ls -l /

Navigate to Home Directory

cd /home

Display users:

ls

Check Current Directory

pwd

Possible output:

/home/student

Explore `/etc`

cd /etc
ls

Configuration files and directories can be viewed here.

Explore `/var`

cd /var
ls

The `/var` directory contains changing system data such as logs and caches.

Explore `/proc`

cd /proc
ls

The `/proc` directory provides information about running processes and the kernel. It is a **virtual file system**, not a conventional disk directory.

Result

The major directories of the Linux file-system hierarchy are successfully identified and explored.

Part E: Linux File Systems

4. Study and Identify Linux File Systems

Definition

A **file system** defines how files and directories are organized, stored, and accessed on a storage device.

RHEL commonly uses **XFS** as the default file system for many installations, although the exact configuration depends on the RHEL version and installation choices.

Practical 1: Display Mounted File Systems

Execute:

df -Th

Example Output

```
Filesystem     Type   Size  Used Avail Use% Mounted on
/dev/sda3      xfs     30G   10G   20G  34% /
/dev/sda2      xfs    1.0G  200M  800M  20% /boot
```

The actual device names, sizes, and file-system types will vary.

Explanation

- `df` → displays disk-space usage

- `-T` → displays file-system type
- `-h` → displays sizes in human-readable format

Practical 2: Display Block Devices

Execute:

lsblk

For file-system information:

lsblk -f

Example

NAME FSTYPE MOUNTPOINT
sda
├─sda1 xfs /boot
└─sda2 xfs /

The exact output depends on how the VM was partitioned.

Practical 3: Check Disk Space

Execute:

df -h

This displays:

- Total disk space
- Used space
- Available space
- Percentage used
- Mount point

Practical 4: Check File-System Mounts

Execute:

mount

For a more readable overview:

findmnt

This displays the relationship between devices, file systems, and mount points.

Part F: Basic Linux File and Directory Commands

The following commands can be performed to understand the Linux file system practically.

1. Create a Directory

mkdir linux_practical

2. Enter the Directory

cd linux_practical

3. Create a File

touch test.txt

4. Display Files

ls

5. Write Data to File

echo "Linux Practical" > test.txt

6. Display File Contents

cat test.txt

Output:

Linux Practical

7. Copy a File

cp test.txt copy.txt

8. Rename a File

mv copy.txt renamed.txt

9. Delete a File

rm renamed.txt

10. Remove Directory

First move outside the directory:

cd ..

Then:

rm -r linux_practical

> **Caution:** `rm` permanently removes files in the normal shell workflow. Avoid using `rm -rf` unless you fully understand the target path.

Part G: Complete Practical Task

Problem Statement

Install RHEL in a VirtualBox/VMware virtual machine and perform the following operations:

1. Identify the display/login manager.
2. Identify the desktop environment.
3. Display the Linux directory structure.
4. Identify the file-system type.
5. Create a directory named `RHEL_Practical`.
6. Create a file named `student.txt`.
7. Write text into the file.
8. Display the file contents.
9. Display disk and file-system information.

Solution

Execute the following commands sequentially:

```
# Display Linux version
cat /etc/redhat-release

# Display kernel version
uname -r

# Identify desktop environment
echo $XDG_CURRENT_DESKTOP

# Check display manager
systemctl status display-manager

# Go to root directory
cd /

# Display directory structure
ls -l

# Go to user's home directory
cd ~

# Create practical directory
mkdir RHEL_Practical
```

```
# Enter directory
cd RHEL_Practical

# Create a file
touch student.txt

# Write data
echo "Red Hat Enterprise Linux Practical" > student.txt

# Display file
cat student.txt

# Display current directory
pwd

# Display files
ls -l

# Display file-system information
df -Th

# Display block-device information
lsblk -f
```

Expected Important Results

```
Desktop Environment : GNOME
Display Manager     : GDM
File System         : XFS (if selected/default for the installation)
Current Directory   : /home/student/RHEL_Practical
File                : student.txt
```

Note: The exact desktop, display manager, partition names, and file-system type can vary with the RHEL release and installation choices.

Viva Questions with Answers

1. What is RHEL?

Answer: RHEL stands for **Red Hat Enterprise Linux**, an enterprise Linux operating system developed by Red Hat.

2. What is VirtualBox?

Answer: VirtualBox is virtualization software that allows operating systems to run inside virtual machines.

3. What is VMware?

Answer: VMware provides virtualization software used to create and run virtual machines.

4. What is a display manager?

Answer: A display manager provides the graphical login interface and starts the user's graphical session.

5. Which display manager is commonly used with GNOME on RHEL?

Answer: GDM (GNOME Display Manager).

6. What is a desktop environment?

Answer: A desktop environment provides the graphical user interface and applications used to interact with the operating system.

7. Which desktop environment is commonly used in RHEL?

Answer: GNOME.

8. What does `/` represent in Linux?

Answer: `/` represents the **root directory**, the top of the Linux file-system hierarchy.

9. What is the purpose of `/home`?

Answer: It contains the home directories of normal users.

10. What is the purpose of `/etc`?

Answer: It contains system-wide configuration files.

11. What is the purpose of `/var`?

Answer: It stores variable data such as logs, caches, and spool files.

12. What is `/proc`?

Answer: `/proc` is a virtual file system that provides information about processes and the running kernel.

13. Which command displays disk usage?

Answer:

df -h

14. Which command displays block devices?

Answer:

lsblk

15. Which command displays the current directory?

Answer:

pwd

16. Which command lists files?

Answer:

ls

17. Which command creates a directory?

Answer:

mkdir

18. Which command creates an empty file?

Answer:

touch

19. Which command displays the contents of a file?

Answer:

cat

20. What is XFS?

Answer: XFS is a high-performance journaling file system commonly used by RHEL installations.

Final Result

RHEL was successfully installed in a VirtualBox/VMware virtual machine. The Linux graphical environment was studied by identifying the display/login manager and desktop environment. The Linux directory hierarchy and major file systems were explored using commands such as `ls`, `df`, `lsblk`, `findmnt`, and `pwd`. Basic file and directory operations were also successfully performed.

Linux Practical 2: Basic System Commands

Aim

To study and perform basic Linux system commands involving **input/output redirection, pipes, `grep`, regular expressions, user switching, and text-file processing commands** such as `cat`, `cut`, `less`, `head`, `tail`, `sort`, `wc`, `grep`, and `awk`.

A. Input-Output Redirection

1. Output Redirection `>`

The `>` operator redirects standard output to a file. If the file already exists, its contents are **overwritten**.

Practical

echo "Welcome to Linux" > message.txt

Display the file:

cat message.txt

Output

Welcome to Linux

Now execute:

echo "Red Hat Enterprise Linux" > message.txt
cat message.txt

Output

Red Hat Enterprise Linux

The previous content has been replaced.

Result

Output is successfully redirected from the terminal to a file.

---

2. Append Redirection `>>`

The >> operator adds output to the **end of an existing file** without deleting its previous contents.

Practical

```
echo "Linux is an operating system" > notes.txt
echo "Linux supports multiple users" >> notes.txt
echo "Linux provides powerful commands" >> notes.txt
```

Display the file:

```
cat notes.txt
```

Output

```
Linux is an operating system
Linux supports multiple users
Linux provides powerful commands
```

Difference Between > and >>

Operator	Function
>	Creates/overwrites a file
>>	Creates/appends to a file

---

3. Input Redirection <

The < operator takes input from a file instead of the keyboard.

Practical

Create a file:

```
cat > students.txt
```

Enter:

```
Amit
Ravi
Priya
Sneha
```

Press **Ctrl+D**.

Now:

```
wc -l < students.txt
```

Output

4

Here, `wc` receives its input from `students.txt`.

---

4. Error Redirection `2>`

Linux uses:

- `0` → Standard input
- `1` → Standard output
- `2` → Standard error

The `2>` operator redirects error messages to a file.

Practical

Run:

ls /invalid_directory 2> error.txt

Display the error file:

cat error.txt

Possible output:

ls: cannot access '/invalid_directory': No such file or directory

The error message is stored in `error.txt` instead of being displayed on the terminal.

---

5. Redirect Output and Error Together

To redirect both standard output and standard error:

command > output.txt 2>&1

Example:

ls /home /invalid > result.txt 2>&1

Both successful output and error messages are stored in `result.txt`.

In Bash, another convenient form is:

ls /home /invalid &> result.txt

---

6. Pipe |

A pipe sends the output of one command as input to another command.

Syntax

command1 | command2

Example

ls -l | less

The output of `ls -l` is passed to `less`.

Another Example

cat /etc/passwd | grep bash

This displays only lines containing `bash`.

Example with `wc`

ls | wc -l

This counts the number of entries displayed by `ls`.

Result

Input/output redirection and pipelines are successfully used.

B. Using `grep` and Regular Expressions

1. `grep` Command

`grep` searches text for a specified pattern.

Syntax

grep [options] pattern filename

Create a file:

cat > employees.txt

Enter:

Amit Sales 25000
Ravi IT 35000
Priya HR 30000
Sneha IT 40000
Rahul Sales 28000

Press **Ctrl+D**.

---

2. Search for a Word

grep "IT" employees.txt

Output

Ravi IT 35000
Sneha IT 40000

---

3. Case-Insensitive Search

grep -i "it" employees.txt

`-i` ignores uppercase/lowercase differences.

---

4. Display Line Numbers

grep -n "Sales" employees.txt

Possible output:

1:Amit Sales 25000
5:Rahul Sales 28000

---

5. Invert Search

The `-v` option displays lines that **do not** match the pattern.

grep -v "IT" employees.txt

Output

Amit Sales 25000
Priya HR 30000
Rahul Sales 28000

Regular Expressions

Regular expressions allow `grep` to search for patterns rather than only exact words.

Common Regular Expression Symbols

Pattern	Meaning	Example
`^`	Beginning of line	`^Amit`
`$`	End of line	`40000$`
`.`	Any single character	`R.vi`
`*`	Zero or more occurrences	`A*`
`[abc]`	One character from set	`[AH]`
`[0-9]`	Any digit	`[0-9]`
`[A-Z]`	Uppercase letter	`[A-Z]`
`^$`	Empty line	`^$`

---

1. Search at Beginning of Line

grep "^Amit" employees.txt

This finds lines beginning with `Amit`.

---

2. Search at End of Line

grep "40000$" employees.txt

This finds lines ending with `40000`.

---

3. Search Using Character Class

grep "[0-9]" employees.txt

This finds lines containing digits.

---

4. Extended Regular Expressions

Use `grep -E` for extended regular expressions.

grep -E "IT|HR" employees.txt

This searches for either `IT` or `HR`.

Result

Text is successfully analyzed using `grep` and regular expressions.

C. Log In and Switch Users in Multi-User Targets

1. Check Current User

whoami

Example:

student

Another command:

id

It displays the user's UID, GID, and group information.

---

2. View Logged-In Users

who

or:

w

`who` shows users currently logged in.

`w` provides additional information such as login time and current activity.

---

3. Switch to Another User

The su command means **substitute user**.

Syntax

su - username

Example:

su - student2

Enter the password when prompted.

Verify:

whoami

Output:

student2

To return to the previous user:

exit

---

4. Switch to Root User

On systems where root login is configured and permitted:

su -

Enter the root password.

Check:

whoami

Output:

root

Exit:

exit

Important

On RHEL systems, administrative tasks are commonly performed using sudo where it has been configured:

sudo command

For example:

sudo systemctl status sshd

---

5. Multi-User Targets

RHEL uses **systemd targets** to define system operating states.

Check the default target:

systemctl get-default

Typical graphical systems may return:

graphical.target

Display the current target:

systemctl list-units --type=target

Common Targets

Target	Purpose
`multi-user.target`	Non-graphical multi-user system
`graphical.target`	Multi-user system with graphical interface
`rescue.target`	Rescue/maintenance mode
`emergency.target`	Minimal emergency environment

Switch to Multi-User Target

The following command requires appropriate administrative privileges:

sudo systemctl isolate multi-user.target

This changes the current system state to the non-graphical multi-user target.

To switch back to graphical mode:

sudo systemctl isolate graphical.target

Caution

Do not perform `systemctl isolate` on a system you are remotely connected to unless you understand the consequences, because services required for your connection may be stopped.

Result

User identity, logged-in users, user switching, and systemd multi-user targets are successfully studied.

D. Working with Text Files

The following commands are important for creating, viewing, filtering, sorting, and analyzing text files.

---

1. `cat`

`cat` displays the contents of a file.

Example

cat employees.txt

Create a File Using `cat`

cat > students.txt

Enter:

101 Amit BCA
102 Ravi BSc
103 Priya BCA
104 Sneha BBA

Press:

Ctrl+D

Display:

cat students.txt

---

2. `less`

`less` displays a file one screen at a time.

less /etc/passwd

Useful keys:

Key	Function
`Space`	Next page
`b`	Previous page
`/word`	Search
`q`	Quit

For large files, `less` is more convenient than `cat`.

---

3. `head`

`head` displays the beginning of a file.

head students.txt

Display the first 2 lines:

head -n 2 students.txt

---

4. `tail`

`tail` displays the end of a file.

tail students.txt

Display the last 2 lines:

tail -n 2 students.txt

Monitor a Log File

tail -f /var/log/messages

This continuously displays new lines added to the file.

> Access to particular log files depends on permissions and the RHEL configuration.

---

5. `sort`

`sort` arranges lines in alphabetical order.

sort students.txt

Sort Numerically

For a numeric field, `sort -n` can be used appropriately.

Example:

cat > marks.txt
75
45
90
60
80

Press **Ctrl+D**.

Execute:

sort -n marks.txt

Output:

45
60
75
80
90

Reverse order:

sort -nr marks.txt

Output:

90
80
75
60
45

---

6. `wc`

wc means **word count**. It can count lines, words, and bytes/characters.

Count Lines

wc -l students.txt

Count Words

wc -w students.txt

Count Bytes

wc -c students.txt

Complete Information

wc students.txt

Typical output has:

lines words bytes filename

---

7. `grep`

Search for BCA students:

grep "BCA" students.txt

Output:

101 Amit BCA
103 Priya BCA

Count matching lines:

grep -c "BCA" students.txt

---

8. `cut`

`cut` extracts selected portions of each line.

Suppose `students.csv` contains:

101,Amit,BCA,75
102,Ravi,BSc,82
103,Priya,BCA,91
104,Sneha,BBA,68

Display only student names:

cut -d',' -f2 students.csv

Output:

Amit
Ravi
Priya
Sneha

Display course:

cut -d',' -f3 students.csv

Output:

BCA
BSc
BCA
BBA

Explanation

- `-d','` → comma is the delimiter
- `-f2` → select field 2

---

9. awk

`awk` is a powerful text-processing and data-analysis command.

Using `students.csv`:

101,Amit,BCA,75
102,Ravi,BSc,82
103,Priya,BCA,91
104,Sneha,BBA,68

Display Student Names

awk -F',' '{print $2}' students.csv

Output:

Amit
Ravi
Priya
Sneha

Display Name and Marks

awk -F',' '{print $2, $4}' students.csv

Output:

Amit 75
Ravi 82
Priya 91
Sneha 68

Display Students Scoring More Than 80

awk -F',' '$4 > 80 {print $2, $4}' students.csv

Output:

Ravi 82
Priya 91

Calculate Average Marks

awk -F',' '{sum += $4; count++} END {print "Average =", sum/count}' students.csv

Output:

Average = 79

E. Combined Practical Using Pipes

The real power of Linux commands comes from combining them.

Example 1: Find BCA Students

grep "BCA" students.csv

Example 2: Extract BCA Students' Names

grep "BCA" students.csv | cut -d',' -f2

Output:

Amit
Priya

Example 3: Sort Student Names

cut -d',' -f2 students.csv | sort

Output:

Amit
Priya
Ravi
Sneha

Example 4: Count BCA Students

grep "BCA" students.csv | wc -l

Output:

2

Example 5: Display Top 2 Students by Marks

sort -t',' -k4,4nr students.csv | head -n 2

Output:

103,Priya,BCA,91
102,Ravi,BSc,82

Explanation

- `-t','` → comma is the field separator
- `-k4,4` → sort according to field 4
- `n` → numerical sorting
- `r` → reverse order
- `head -n 2` → display first two records

F. Integrated Practical Exercise

Problem Statement

Create a student data file and use Linux commands to:

1. Create a text file.
2. Display its contents.
3. Display the first and last records.
4. Extract student names.
5. Sort the names.
6. Search for BCA students.
7. Count BCA students.
8. Display students scoring above 80.
9. Redirect output to another file.
10. Use pipes to combine commands.

Solution

Create the file:

cat > students.csv

Enter:

101,Amit,BCA,75
102,Ravi,BSc,82
103,Priya,BCA,91
104,Sneha,BBA,68
105,Rahul,BCA,88
106,Neha,BSc,79

Press **Ctrl+D**.

Step 1: Display File

cat students.csv

Step 2: First Three Records

head -n 3 students.csv

Step 3: Last Two Records

tail -n 2 students.csv

Step 4: Extract Names

cut -d',' -f2 students.csv

Step 5: Sort Names

cut -d',' -f2 students.csv | sort

Step 6: Find BCA Students

grep "BCA" students.csv

Step 7: Count BCA Students

grep -c "BCA" students.csv

Output:

3

Step 8: Find Students Scoring Above 80

awk -F',' '$4 > 80 {print $2, $4}' students.csv

Output:

Ravi 82
Priya 91
Rahul 88

Step 9: Save BCA Students to a File

grep "BCA" students.csv > bca_students.txt

Check:

cat bca_students.txt

Step 10: Append More Information

echo "End of BCA Student List" >> bca_students.txt

Step 11: Count Records

wc -l students.csv

Output:

6 students.csv

Step 12: Find Highest Marks

sort -t',' -k4,4nr students.csv | head -n 1

Output:

103,Priya,BCA,91

G. Practical Summary Table

Command	Purpose	Example
`cat`	Display/create text	`cat file.txt`
`less`	View large files page-wise	`less file.txt`
`head`	Display beginning	`head -n 5 file.txt`
`tail`	Display end	`tail -n 5 file.txt`
`sort`	Sort lines	`sort file.txt`
`wc`	Count lines/words/bytes	`wc -l file.txt`
`grep`	Search patterns	`grep "Linux" file.txt`
`cut`	Extract fields/characters	`cut -d',' -f2 file.csv`
`awk`	Process structured text	`awk -F',' '{print $2}' file.csv`
`>`	Redirect/overwrite output	`ls > list.txt`
`>>`	Append output	`date >> log.txt`

`<`	Take input from file	`wc -l < file.txt`
`2>`	Redirect errors	`cmd 2> error.txt`
`|`	Pipe commands	`cat file | grep Linux`
`whoami`	Show current user	`whoami`
`who`	Show logged-in users	`who`
`su`	Switch user	`su - student2`
`id`	Show user/group IDs	`id`

Result

The basic Linux system commands were successfully studied and executed. Input/output redirection using `>`, `>>`, `<`, and `2>` was performed. Pipes were used to combine commands. Text was analyzed using `grep` and regular expressions. User login and switching were studied using `who`, `whoami`, `id`, and `su`. Text files were created, viewed, searched, sorted, filtered, and analyzed using `cat`, `less`, `head`, `tail`, `sort`, `wc`, `grep`, `cut`, and `awk`.

Linux Practical 3: Essential File Management Tools

Aim

To study and perform essential Linux file-management operations, including:

- Creating, deleting, copying, and moving files/directories
- Creating and managing hard and soft links
- Archiving and compressing files using `tar`, `star`, `gzip`, and `bzip2`
- Managing file ownership
- Managing basic and advanced permissions
- Using `ugo/rwx` permissions
- Creating set-GID collaborative directories
- Managing ACLs
- Configuring default permissions using `umask`
- Working with user-extended file attributes

---

1. Create, Delete, Copy and Move Files

A. Create a Directory

Use `mkdir` to create a directory.

mkdir linux_files

Check:

ls -ld linux_files

Create Nested Directories

mkdir -p project/data/reports

The `-p` option creates parent directories if they do not already exist.

B. Create a File

Use `touch`:

touch file1.txt

Create multiple files:

touch file2.txt file3.txt file4.txt

Verify:

ls

---

2. Delete Files and Directories

Delete a File

rm file1.txt

Delete Multiple Files

rm file2.txt file3.txt

Delete an Empty Directory

rmdir linux_files

Delete a Directory and Its Contents

rm -r project

> **Caution:** `rm` normally removes files without moving them to a recycle bin. Always verify the path before using recursive deletion.

---

3. Copy Files and Directories

Copy a File

cp file1.txt backup.txt

Verify:

ls

Copy File to a Directory

mkdir backup
cp file1.txt backup/

Copy a Directory

Use `-r`:

cp -r backup backup_copy

Preserve File Attributes

cp -p file1.txt backup/

---

4. Move and Rename Files

The `mv` command is used both to **move** and **rename** files.

Rename a File

mv file1.txt document.txt

Move a File

mv document.txt backup/

Move Multiple Files

mv file2.txt file3.txt backup/

Rename a Directory

mv backup backup_files

---

5. Complete File Management Practical

Execute:

mkdir file_practical
cd file_practical

touch file1.txt file2.txt file3.txt

mkdir documents

cp file1.txt documents/

mv file2.txt documents/file2_new.txt

ls -l
ls -l documents

Expected Structure

file_practical/
├── file1.txt
├── file3.txt
└── documents/
  ├── file1.txt

└── file2_new.txt

---

6. Working with Links

Linux supports two important types of links:

1. **Hard links**
2. **Symbolic/soft links**

A. Hard Link

A hard link is another directory entry pointing to the same underlying inode/data as the original file.

Create a File

echo "Linux File Management" > original.txt

Create Hard Link

ln original.txt hardlink.txt

Check inode numbers:

ls -li original.txt hardlink.txt

Example:

123456 -rw-r--r-- 2 student student 22 original.txt
123456 -rw-r--r-- 2 student student 22 hardlink.txt

Both files have the **same inode number**.

Modify Through Hard Link

echo "Hard link example" >> hardlink.txt

Now:

cat original.txt

The change is visible through `original.txt` because both names refer to the same file data.

Remove Original

rm original.txt

The hard link remains:

cat hardlink.txt

The data is still accessible.

Important Characteristics of Hard Links

- Point to the same inode.
- Usually cannot cross file-system boundaries.
- Normally cannot be created for directories by ordinary users.
- Data remains available as long as at least one hard link remains.

B. Soft Link / Symbolic Link

A symbolic link is a separate file containing a reference/path to another file or directory.

Create a Soft Link

echo "Symbolic Link Example" > source.txt
ln -s source.txt softlink.txt

Check:

ls -l source.txt softlink.txt

Example:

-rw-r--r-- 1 student student 22 source.txt
lrwxrwxrwx 1 student student 10 softlink.txt -> source.txt

The `l` at the beginning indicates a symbolic link.

Read Through the Link

cat softlink.txt

Remove Source

rm source.txt

Now:

cat softlink.txt

The symbolic link becomes a **dangling/broken link** because its target no longer exists.

Hard Link vs Soft Link

Feature	Hard Link	Soft Link

Command	`ln file link`	`ln -s file link`
Inode	Same inode	Different inode
Can link directories normally?	No	Yes
Cross file systems	No	Yes
If original is deleted	Link still works	Link becomes broken
Stores target path	No	Yes
Common use	Additional name for same file	Shortcut/reference

---

7. Working with Archives

An **archive** combines multiple files/directories into a single archive file.

`tar` is the standard archive utility commonly used on RHEL.

A. Create a TAR Archive

Create sample files:

```
mkdir project
echo "File One" > project/file1.txt
echo "File Two" > project/file2.txt
echo "File Three" > project/file3.txt
```

Create archive:

```
tar -cf project.tar project/
```

Explanation

- `c` → create archive
- `f` → specify archive file

Check:

```
ls -lh project.tar
```

B. List Contents of TAR Archive

tar -tf project.tar

Possible output:

project/
project/file1.txt
project/file2.txt
project/file3.txt

C. Extract TAR Archive

Create an extraction directory:

mkdir extracted

Extract:

tar -xf project.tar -C extracted/

Check:

find extracted -type f

Explanation

- `x` → extract
- `f` → archive file
- `-C` → extract into specified directory

---

8. Compress Using gzip

`gzip` compresses individual files.

Compress a File

gzip file1.txt

The file becomes:

file1.txt.gz

Decompress

gzip -d file1.txt.gz

or:

gunzip file1.txt.gz

---

9. Create a Compressed TAR Archive Using gzip

tar -czf project.tar.gz project/

Options

- `c` → create
- `z` → gzip compression
- `f` → archive file

Check contents:

tar -tzf project.tar.gz

Extract:

tar -xzf project.tar.gz

---

10. Compress Using bzip2

`bzip2` provides another compression method.

Compress

bzip2 file2.txt

Result:

file2.txt.bz2

Decompress

bzip2 -d file2.txt.bz2

or:

bunzip2 file2.txt.bz2

---

11. Create a TAR+bzip2 Archive

tar -cjf project.tar.bz2 project/

List Contents

tar -tjf project.tar.bz2

Extract

tar -xjf project.tar.bz2

---

12. Using `star`

`star` is an alternative archive utility with advanced features and is encountered in some Unix/Linux environments.

Check Installation

command -v star

If available, create an archive:

star -c -f=project.star project/

List contents:

star -t -f=project.star

Extract:

mkdir star_extract
cd star_extract
star -x -f=../project.star

Important

`star` may not be installed by default on every RHEL installation. If the command is unavailable, install the appropriate package through your organization's configured RHEL repositories.

---

13. Archive and Compression Comparison

Command	Purpose
`tar`	Archive files/directories
`gzip`	Compress a file
`bzip2`	Compress a file
`star`	Alternative/advanced archive utility
`tar -czf`	Create gzip-compressed TAR
`tar -cjf`	Create bzip2-compressed TAR

`tar -xzf` Extract `.tar.gz`

`tar -xjf` Extract `.tar.bz2`

Common Extensions

.tar → archive
.tar.gz → TAR + gzip
.tgz → TAR + gzip
.tar.bz2 → TAR + bzip2
.gz → gzip-compressed file
.bz2 → bzip2-compressed file

---

14. Managing File Ownership

Linux associates each file with:

- User/owner
- Group
- Permissions

Check ownership:

ls -l file1.txt

Example:

-rw-r--r-- 1 student students 25 Aug 29 09:00 file1.txt

Here:

Owner = student
Group = students

A. Change File Owner

The `chown` command changes ownership.

For example, with appropriate administrative privileges:

sudo chown student file1.txt

Change Owner and Group

sudo chown student:students file1.txt

Change Group Only

Use `chgrp`:

sudo chgrp students file1.txt

Verify:

ls -l file1.txt

> Changing ownership generally requires root privileges or appropriate delegated privileges.

---

15. Understanding Basic `ugo/rwx` Permissions

Linux permissions are divided into three classes:

- `u` → user/owner
- `g` → group
- `o` → others

Each class can have:

- `r` → read
- `w` → write
- `x` → execute

Example:

-rwxr-xr--

Breakdown:

```
- rwx r-x r--
  |   |   |
  |   |   └── Others
  |   └────── Group
  └────────── Owner
```

Meaning

Permission	Value
Read (`r`)	4
Write (`w`)	2
Execute (`x`)	1

Therefore:

rwx = 4 + 2 + 1 = 7
r-x = 4 + 0 + 1 = 5
r-- = 4 + 0 + 0 = 4

So:

rwxr-xr-- = 754

---

16. List Permissions

ls -l file1.txt

For a directory:

ls -ld project

---

17. Set Permissions Using Symbolic Mode

Give owner execute permission:

chmod u+x file1.txt

Remove write permission from group:

chmod g-w file1.txt

Give read permission to others:

chmod o+r file1.txt

Set permissions for all three classes:

chmod u=rwx,g=rx,o=r file1.txt

---

18. Set Permissions Using Numeric Mode

Set:

Owner = rwx
Group = r-x
Others = r--

Use:

chmod 754 file1.txt

Verify:

ls -l file1.txt

---

19. Directory Permissions

For directories:

- `r` → list directory entries
- `w` → create/delete/rename entries, subject to other permission rules
- `x` → traverse/search the directory

Example:

mkdir testdir
chmod 755 testdir
ls -ld testdir

---

20. Advanced Permissions

Linux provides special permission bits:

1. **setuid**
2. **setgid**
3. **sticky bit**

These are represented by special execute positions.

A. Set-GID

The **set-GID** bit on a directory causes newly created files/subdirectories to inherit the directory's group, which is useful for team collaboration.

Create Group

With administrative privileges:

sudo groupadd projectteam

Create Collaboration Directory

sudo mkdir /project

Change Group

sudo chown :projectteam /project

Give Group Collaboration Permissions

sudo chmod 2775 /project

Check:

ls -ld /project

Expected permission pattern:

drwxrwsr-x

The s in the group execute position indicates **set-GID**.

---

21. Test Set-GID Directory

Add users to the group as appropriate:

sudo usermod -aG projectteam student

After the user starts a new login session, create a file:

cd /project
touch report.txt

Check:

ls -l report.txt

The file should inherit the directory's group:

projectteam

Why Set-GID Is Useful

Suppose several employees work on:

/project

Without set-GID, files may inherit different primary groups.

With set-GID:

/project
  ↓
projectteam group
  ↓
New files inherit projectteam

This simplifies collaborative access management.

---

22. Sticky Bit

The sticky bit is commonly used on shared directories such as `/tmp`.

Set it on a test directory:

mkdir shared
chmod 1777 shared

Check:

ls -ld shared

Possible result:

drwxrwxrwt

The `t` indicates the sticky bit.

It restricts deletion/renaming in a shared writable directory so that users generally cannot remove or rename files belonging to other users, subject to ownership and administrative privileges.

---

23. Managing ACLs

What is ACL?

ACL (Access Control List) provides more granular permissions than traditional owner/group/other permissions.

For example, traditional permissions allow:

Owner
Group
Others

ACLs allow additional named users and groups.

---

24. Check ACL

First create a file:

touch report.txt

Display ACL:

getfacl report.txt

Example:

```
# file: report.txt
# owner: student
# group: students
user::rw-
group::r--
other::r--
```

---

25. Set an ACL for a User

Suppose user `ravi` should receive read and write permission:

sudo setfacl -m u:ravi:rw report.txt

Check:

getfacl report.txt

You may see:

```
user::rw-
user:ravi:rw-
group::r--
other::r--
```

---

26. Set ACL for a Group

sudo setfacl -m g:projectteam:rwx report.txt

Check:

getfacl report.txt

---

27. Remove a User ACL

sudo setfacl -x u:ravi report.txt

Verify:

getfacl report.txt

---

28. Remove All Extended ACLs

setfacl -b report.txt

The `-b` option removes extended ACL entries.

---

29. ACL Mask

The ACL mask controls the effective permissions for named users, named groups, and the owning group.

Set a mask:

setfacl -m m:rw report.txt

View:

getfacl report.txt

Look for:

mask::rw-

---

30. Default ACL

Default ACLs can be applied to directories so that newly created files/directories inherit ACL entries.

Create directory:

mkdir shared_project

Set default ACL for user `ravi`:

sudo setfacl -d -m u:ravi:rwx shared_project

View:

getfacl shared_project

You should see entries beginning with:

default:

Create a file:

touch shared_project/file.txt

Check:

getfacl shared_project/file.txt

The inherited ACL depends on the requested mode and ACL mask.

---

31. Setting Default Permissions with `umask`

What is `umask`?

`umask` determines which permission bits are **removed from the permissions requested when new files and directories are created**.

Check current value:

umask

Example:

0022

---

32. Test `umask`

Create a file:

touch test.txt

Check:

ls -l test.txt

Create a directory:

mkdir testdir

Check:

ls -ld testdir

Typical default requested modes are:

Files → 666
Directories → 777

The actual permissions are reduced by the `umask`.

For example, with a common `0022` mask:

File: 666 - 022 → 644
Directory: 777 - 022 → 755

This is conceptual permission masking rather than ordinary arithmetic subtraction for every bit.

---

33. Temporarily Change `umask`

Set:

umask 027

Check:

umask

Create:

touch secure.txt
mkdir secure_dir

Check:

ls -l secure.txt
ls -ld secure_dir

Typical result:

secure.txt → 640
secure_dir → 750

The exact result can also be affected by application behavior and ACL defaults.

---

34. Make `umask` Persistent

A user's shell configuration can contain:

umask 027

For a Bash login environment, appropriate shell startup files can be used according to the system's configuration and administrative policy.

After starting a new session:

umask

---

35. User-Extended Attributes

Extended attributes (**xattrs**) store metadata associated with files beyond normal permissions, ownership, and timestamps.

On Linux, `getfattr` and `setfattr` are commonly used.

Check availability:

command -v getfattr
command -v setfattr

---

36. Set an Extended Attribute

Create a file:

touch document.txt

Set an attribute:

setfattr -n user.description -v "Linux Practical File" document.txt

Display:

getfattr -d document.txt

Possible output:

file: document.txt
user.description="Linux Practical File"

---

37. Read a Specific Extended Attribute

getfattr -n user.description document.txt

---

38. Remove Extended Attribute

setfattr -x user.description document.txt

Verify:

getfattr -d document.txt

---

39. Complete Integrated Practical

Problem Statement

Create a collaborative Linux project directory and demonstrate:

1. File and directory creation
2. Copying and moving
3. Hard and soft links
4. TAR archiving
5. gzip compression
6. bzip2 compression
7. Ownership management
8. Basic permissions
9. Set-GID collaboration
10. ACL management
11. `umask`
12. Extended attributes

Solution

Step 1: Create Project

mkdir ~/LinuxProject
cd ~/LinuxProject

Step 2: Create Files

echo "Linux Administration" > file1.txt
echo "File Management Practical" > file2.txt

Step 3: Create Backup Directory

mkdir backup

Step 4: Copy

cp file1.txt backup/

Step 5: Move

mv file2.txt backup/

Step 6: Create Hard Link

ln file1.txt file1_hardlink.txt

Step 7: Create Soft Link

ln -s file1.txt file1_softlink.txt

Check:

ls -li

Step 8: Create TAR Archive

tar -cf LinuxProject.tar .

Check:

tar -tf LinuxProject.tar

Step 9: Create gzip Archive

tar -czf LinuxProject.tar.gz backup/

Check:

tar -tzf LinuxProject.tar.gz

Step 10: Create bzip2 Archive

tar -cjf LinuxProject.tar.bz2 backup/

Check:

tar -tjf LinuxProject.tar.bz2

Step 11: Set Basic Permissions

chmod 640 file1.txt

Check:

ls -l file1.txt

Expected:

-rw-r-----

Step 12: Add Extended Attribute

setfattr -n user.project -v "Linux Administration" file1.txt

Check:

getfattr -d file1.txt

Step 13: Set ACL

Give user `ravi` read/write permission:

sudo setfacl -m u:ravi:rw file1.txt

Check:

getfacl file1.txt

Step 14: Create Collaboration Directory

sudo mkdir /LinuxTeam

Create a group:

sudo groupadd linuxteam

Change group:

sudo chown :linuxteam /LinuxTeam

Set set-GID permissions:

sudo chmod 2775 /LinuxTeam

Check:

ls -ld /LinuxTeam

Expected pattern:

drwxrwsr-x

Step 15: Add ACL to Collaboration Directory

Give `ravi` full access:

sudo setfacl -m u:ravi:rwx /LinuxTeam

Check:

getfacl /LinuxTeam

Step 16: Set Default ACL

sudo setfacl -d -m g:linuxteam:rwx /LinuxTeam

Check:

getfacl /LinuxTeam

Step 17: Test `umask`

umask

Set temporarily:

umask 027

Create:

touch umask_file.txt
mkdir umask_directory

Check:

ls -l umask_file.txt
ls -ld umask_directory

---

40. Important Command Cheat Sheet

Task	Command
Create file	`touch file`
Create directory	`mkdir dir`
Create nested directories	`mkdir -p a/b/c`
Delete file	`rm file`
Delete directory	`rm -r dir`
Copy file	`cp file destination/`
Copy directory	`cp -r dir destination/`
Move/rename	`mv old new`
Create hard link	`ln file link`
Create soft link	`ln -s file link`
Show inode	`ls -li`

Create TAR	`tar -cf archive.tar dir/`
Extract TAR	`tar -xf archive.tar`
gzip	`gzip file`
Un-gzip	`gunzip file.gz`
TAR + gzip	`tar -czf archive.tar.gz dir/`
Extract TAR + gzip	`tar -xzf archive.tar.gz`
bzip2	`bzip2 file`
Un-bzip2	`bunzip2 file.bz2`
TAR + bzip2	`tar -cjf archive.tar.bz2 dir/`
Extract TAR + bzip2	`tar -xjf archive.tar.bz2`
Change owner	`chown user file`
Change owner/group	`chown user:group file`
Change group	`chgrp group file`
List permissions	`ls -l`
Set permissions	`chmod 755 file`
Set-GID	`chmod 2775 directory`
Sticky bit	`chmod 1777 directory`
View ACL	`getfacl file`
Set ACL	`setfacl -m u:user:rw file`
Remove ACL entry	`setfacl -x u:user file`
Remove extended ACLs	`setfacl -b file`
Default ACL	`setfacl -d -m ... directory`
View umask	`umask`
Set umask	`umask 027`
Set xattr	`setfattr -n user.key -v value file`

View xattr	`getfattr -d file`
Remove xattr	`setfattr -x user.key file`

Viva Questions and Answers

1. What is a hard link?

A hard link is another directory entry referring to the same inode and file data as the original file.

2. What is a soft link?

A soft link is a symbolic reference containing the path to another file or directory.

3. Which command creates a hard link?

ln source link

4. Which command creates a symbolic link?

ln -s source link

5. What is `tar` used for?

`tar` is primarily used to combine multiple files and directories into an archive.

6. What does `tar -czf` do?

It creates a TAR archive and compresses it using gzip.

7. What does `tar -xzf` do?

It extracts a gzip-compressed TAR archive.

8. What is `gzip`?

`gzip` is a compression utility used to reduce the size of files.

9. What is `bzip2`?

`bzip2` is a block-sorting compression utility that provides another compression method.

10. What is file ownership?

File ownership identifies the user and group associated with a file.

11. What does `chmod` do?

`chmod` changes file or directory permission bits.

12. What does `chown` do?

`chown` changes the owner and, optionally, group of a file or directory.

13. What does `rwx` mean?

`rwx` represents read, write, and execute permissions.

14. What are `ugo`?

`u` means user/owner, `g` means group, and `o` means others.

15. What does permission `755` mean?

Owner = rwx
Group = r-x
Others = r-x

16. What is set-GID on a directory?

Set-GID causes newly created files and subdirectories to inherit the directory's group, which facilitates group collaboration.

17. What is an ACL?

An ACL is an Access Control List that provides fine-grained permissions for additional users and groups.

18. Which commands manage ACLs?

getfacl
setfacl

19. What is `umask`?

`umask` specifies permission bits that are masked out when new files and directories are created.

20. What are extended attributes?

Extended attributes are additional metadata associated with files beyond standard Unix permissions and ownership.

21. Which commands manage extended attributes?

getfattr
setfattr

Result

The essential Linux file-management operations were successfully performed. Files and directories were created, copied, moved, and deleted. Hard and symbolic links were created and examined. Files were archived, compressed, extracted, and uncompressed using `tar`, `star`, `gzip`, and `bzip2`. File ownership and `ugo/rwx` permissions were managed using `chown`, `chgrp`, and `chmod`. Advanced permissions including set-GID and the sticky bit were studied.